University of Geneva

Faculty of Science

Video Compression and Modification Detection

Digital Forensics
14x065

Michel Jean Joseph Donnet

2025-12-06

Contents

1 Introduction 4
2 Methodology 5
3 Redundancies 5
3.1 Psychovisual redundancies 5
3.1.1 Spatial irrelevance 5
3.1.2 Spectral irrelevance 5
3.1.3 Temporal irrelevance 5
3.2 Statistical redundancies 6
3.2.1 Spatial irrelevance 6
3.2.2 Spectral irrelevance 6
3.2.3 Temporal irrelevance 6
4 H.264 7
4.1 Compression 7
4.1.1 Color Space Transform 7
4.1.2 Split into MacroBlocks 7
4.1.3 Partitioning 8
4.1.4 Prediction 8
4.1.4.1 Intra-prediction 8
4.1.4.2 Inter-prediction 8
4.1.5 Residue 9
4.1.6 Transform 10
4.1.7 Quantization 10
4.1.8 Entropy encoding 10
4.2 Decompression 10
5 H265 12
5.1 Compression 12
5.1.1 Partitioning 12
5.1.2 Prediction 13
5.1.3 Transform 13
6 H266 14
6.1 Compression 14
6.1.1 Partitioning 14
6.1.2 Prediction 14
6.1.3 Transform 15
6.1.4 Quantization 15
7 AOMedia Video 1 (AV1) 16
7.1 Compression 16
7.1.1 Partitioning 16
7.1.2 Prediction 17
7.1.3 Transform 17
7.1.4 Quantization 18
7.1.5 Entropy encoding 18
8 Forensic 19

8.1 Partitioning 19
8.1.1 H264 19
8.1.2 H265 19
8.1.3 H266 20
8.1.4 AV1 20
8.2 Prediction 20
8.3 Quantization and Entropy encoding 22
9 Conclusion 23
9.0.1 Note on the use of AI 23
Bibliography 24

1 Introduction

Nowadays, a large number of videos circulate on the internet. Given that each video is composed of around twenty images per second, storing and transmitting videos requires a large amount of data. This is why methods have been implemented to reduce the amount of data used by a video while preserving its visual quality. However, some videos circulating on the internet convey a distorted image of reality through clever modifications to the original content, which can even lead to people being exonerated in court. This is why it is important to know the history of a video, which can be achieved through digital forensics. Since video compression leaves traces, these can be analyzed to reveal any modifications to the video. The objective of this work will therefore be to understand how video compression works and to see how the traces left by compression can be used to detect modifications to the video.

2 Methodology

In general, video are sequence of images called frames. Based on this kind of video, we will start by studying how video compression works. To do this, we will use the h264 codec, as it is the most widely used codec on the web.

3 Redundancies

3.1 Psychovisual redundancies

The imperfections of the human visual system can be exploited to reduce the amount of data used by a video without a loss of visual quality.

3.1.1 Spatial irrelevance

The human visual system has some difficulties to perceive small details due to its limitations. The optics of the eyes and the neural processing tend to smooth out fine patterns. For example: a document printed by a laser printer is composed of small dots that are very close together. However, we cannot see the individual dots at a normal distance: we only perceive a uniform surface.

This is why this feature can be used to remove small details that are invisible or barely visible for human visual system.

3.1.2 Spectral irrelevance

The human visual system consists of approximately 100 million rods responsible for perceiving brightness and approximately 6.5 million cones responsible for perceiving colors. Due to the high amount of rods, the human visual system is more sensitive to brightness compared to color.

There are 3 types of cones:
- L-cones ($\approx 65\%$), sensitive to long wavelengths (such as the red color).
- M-cones ($\approx 33\%$), sensitive to medium wavelengths (such as the green color).
- S-cones ($\approx 2\%$), sensitive to short wavelengths (such as blue color).

In this way, human visual system is less sensitive to blue color than to other colors due to its amount of S-cones.

Therefore, retaining all the spectral details of the video may prove unnecessary for good quality reproduction of the video.

3.1.3 Temporal irrelevance

The human visual system is not sensitive to rapid changes. Thus, in the case of a video, only about 30 frames per second are necessary for humans to perceive smooth motion. Since human visual system is unable to detect rapid changes between successive frames, this can be exploited to reduce the amount of data used by a video without loss of visual quality.

3.2 Statistical redundancies

The statistical redundancies of each frame of a video can be used to optimize the amount of data used. Among these redundancies, we have spatial, spectral, and temporal redundancies, as in the human visual system.

3.2.1 Spatial irrelevance

These redundancies are introduced by a strong correlation between neighboring pixels. By the way, in an image, each pixel is correlated with its neighbors in such a way that the final result is something visible and understandable to humans. An image created with no correlation between pixels is something like a noisy image because each pixel is independent from the others. So the spatial redundancies can be exploited to predict for instance values of pixels according to neighboring pixels. In this manner, some data can be deleted, which reduces amount of data used.

3.2.2 Spectral irrelevance

These redundancies are created by strong correlation between neighboring pixels in color domain. This is because color changes are often gradual, and colored regions regularly extend beyond a single pixel. Thus, a pixel is strongly correlated with its neighbors in the color domain, which can be exploited to reduce amount of data used.

3.2.3 Temporal irrelevance

These redundancies are the most important for video compression. In fact, changes to the elements present in the video occur gradually, especially in consecutive frames. Thus, between two consecutive frames, there will not be many changes, as shown by Figure 1. Indeed, the amount of change created by this moving car is very small.

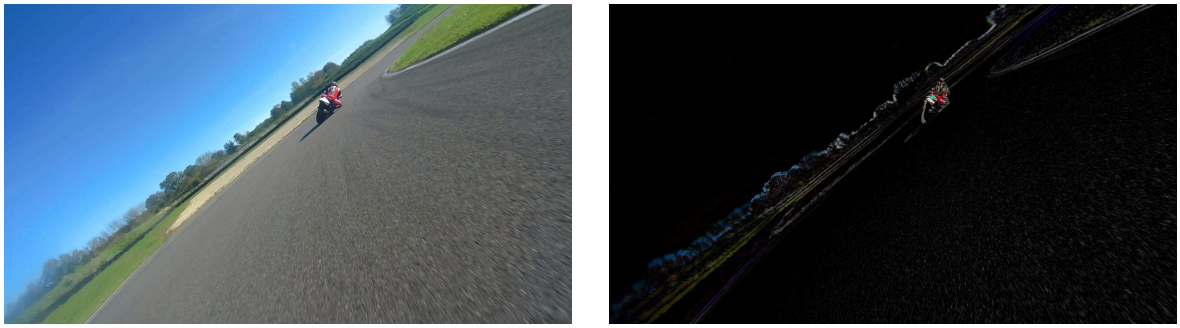


Figure 1: Difference between 2 consecutive frames for a 60 fps video

So if only the changes are recorded, most of the frame can be compressed.

4 H.264

The H.264 codec, also known as MPEG-4 AVC (Advanced Video Coding) or MPEG-4 Part 10, was developed in 2003. It undergoes numerous transformations and is still undergoing improvements today [1].

4.1 Compression

Video compression follows steps similar to those used in image compression, which consist of data transformation, quantization for lossy compression, and data encoding for efficient storage on disk. However, some additional steps are provided to exploit temporal redundancies.

More concretely, we have the steps bellow.

4.1.1 Color Space Transform

The frame is first transformed from RGB color space to YCbCr color space. Instead of representing the frame in RGB color space, the YCbCr color space is used to divide the frame into luminance (Y) and chrominance (both Cb and Cr).

Y channel Cb channel Cr channel

Figure 2: Image decomposed in YCbCr color space

The luminance looks after the luminosity, the brightness of the image whereas the chrominance looks after colors of the image, as shown on Figure 2. On top of that, as we can observe on Figure 2, all structural information is contained in Y channel, which represent clearly the image in grayscale.

In this way, since the human visual system is less sensitive to color than to brightness, chrominance could be compressed more than luminance at a later stage.

4.1.2 Split into MacroBlocks

Then, the frame is divided into MacroBlock generally of size 16×16. These MacroBlocks perfectly represent specific regions of the image and are very useful for working on small areas of the image. This allows for easier detection of elements such as moving and not moving objects, more accurate motion prediction, and an efficient means of compressing data. Indeed, some parts of the image may be flat while others may contain a large amount of detail. And on subsequent frames, some informations can be exactly the same, as represented by the black on Figure 1. In this way, some parts of the image can be compressed more than others.

[1] Wikipedia

4.1.3 Partitioning

Each MacroBlock is then divided into smaller blocks depending on precision needed. This allows a better adaptation on local complexity of the frame.

The MacroBlock can be divided into sub-blocks of size 16×16, 8×8, 8×4, 4×8 or 4×4.

4.1.4 Prediction

This step is divided into two sub-steps.

4.1.4.1 Intra-prediction

In this case, only the current frame is taken into account. The goal is to predict the value of a pixel based on its neighborhood. The prediction mode is chosen by the encoder, such as vertical, horizontal or other modes.

- vertical mode: copy content of previous row
- horizontal mode: copy content of previous column

The prediction that gives the smallest difference between original block frame and predicted block frame is kept according to chosen mode. There exists 9 prediction modes for 4×4 blocks and 4 modes for 16×16 blocks.

The intra-prediction is very useful for scenes without moves.

4.1.4.2 Inter-prediction

In this case, the subsequent frames are used to make the prediction (past and future frames, depending on kind of prediction). The goal is to predict block position according to subsequent frames. This is based on the fact that only small changes are introduced between subsequent frames, as shown on Figure 1.

First of all, we need to introduce a few technical terms, summarized in the Table 1.

I-frame This is an independent frame that is fully encoded using Section 4.1.4.1, which results in low prediction efficiency. The file size is large due to the amount of data that needs to be retained. This type of frame corresponds to the key frames in the video, which are used to encode the other frames.

P-frame This is a frame encoded using the previous I-frame, which results in good prediction efficiency thanks to previous I-frame. The aim is to track the movement of the object in the video and use this to reduce the amount of data used. By exploiting this source of redundancy, the amount of data used is less than that of I-frame, so the compression is better.

B-frame this frame is encoded using the previous I-frame and the following P-frame, which results in high prediction efficiency. By using the previous and following images, the prediction is definitely better, allowing less data to be used than for P-frames. This kind of frame is especially used for compression.

These frames create a Group Of Picture as shown on Figure 6.

Frame type	Dependency	Prediction efficiency	File size	Goal
I-frame	No	Bad	-	Key frame
P-frame	I-frame	Good	+	Track move
B-frame	I-frame, P-frame	High	+++	Compression

Table 1: Kind of frames used in a video

There is no inter-prediction in I-frame, as described in Table 1.

On case of P-frame computation, the block of the P-frame is searched in the previous I-frame. A motion vector is then computed, describing the move of the block between this two frames. Then, the content of the reference block is predicted according to motion vector and I-frame content.

On case of B-frame computation, the block of the B-frame is searched in the previous I-frame and next P-frame. The motion vector is then computed, allowing prediction of the block according to I-frame and P-frame. In this case, the prediction of the block is more accurate thanks to the I-frame and P-frame used, allowing greater data compression later on.

Computing the motion vector is costly, however it allows to achieve great compression on moving objects for P-frames and B-frames.

The structure of the video follows a pattern as shown in the Figure 6. This pattern is called Group Of Picture (GOP) and can be used later for modification detection.

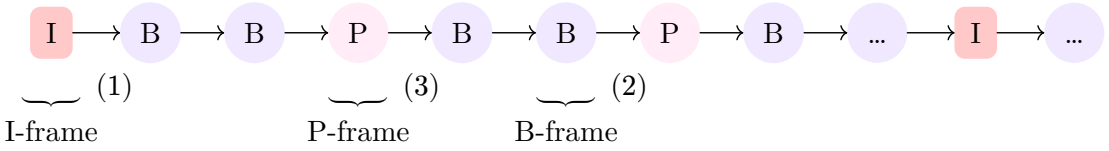


Figure 6: Frame organization in video

4.1.5 Residue

Once the prediction is made for a block, the difference between the predicted block and the original block of the frame is computed. Depending on results, prediction can be reapplied to have smaller residuals allowing better compression, as mentioned in Section 4.1.4.1.

In this way, only the method for establishing the prediction and the residual difference between the prediction and the original image are necessary to reconstruct the original image.

For instance, suppose that we have B_{original} the original block of the frame and $B_{\text{predicted}}$ the predicted block. Then, the residual R can be computed as shown in Equation 4.

$$R = B_{\text{original}} - B_{\text{predicted}} \tag{4}$$

So if only both motion vector for inter-prediction and prediction method for intra-prediction is kept with the residual, the original block can be reconstruct as shown in Equation 5.

$$B_{\text{original}} = R + B_{\text{predicted}} \tag{5}$$

This is why only residual of prediction and information necessary for prediction are retained, allowing for considerable data compression.

4.1.6 Transform

An integer approximation of the Discrete Cosine Transform (DCT) is applied to the prediction residual in order to convert it into frequency coefficients.

The integer approximation is used to facilitate computations and avoid rounding error introduced by a classical DCT that works with floating points. In fact, the entire approximation of the DCT is reversible and only processes integers, which is particularly relevant when working with images and videos in the integer domain.

In this way, the residual is divided into high and low frequencies, which proves useful in the next step: the quantization.

4.1.7 Quantization

The quantization is a lossy step that determine the quality of the compressed video. It is based on imperfection of human visual system, which is less sensitive to small details, as explained in Section 3.1. Since high frequencies represent small details, quantization uses the decomposition of the residue into low and high frequencies to remove the high frequencies. To do this, a quantization matrix is constructed based on the quality factor defined by the user. Then, the residual is divided by the quantization matrix and the result is rounded to have only integer values. Some high frequencies then become 0, which reduces the amount of data to be stored. In this way, some data are lost, that's why this is a lossy step.

4.1.8 Entropy encoding

Finally, the quantified residues and parameters useful for prediction, such as the motion vector or prediction method, are encoded in such a way as to use as few bits as possible and to be able to reconstruct the encoded video. Depending on requirements, two methods can be used:

- Context-adaptive variable-length coding (CAVLC). This is the simplest method, but also the least efficient.
- Context-adaptive binary arithmetic coding (CABAC). This method is more complex, but allows for better compression. This is why it is preferred for high-quality compression.

> **Note**
>
> A very good explanation of how H264 works is available on the website abhik.xyz, which provides interactive explanations.

4.2 Decompression

The process of decompression follows the same steps as video compression, but in reverse order, as shown on Figure 7.

So first, data are decoded. Next, as in the quantization step for encoding, the DCT coefficients were divided by a quantization matrix, the DCT coefficients are multiplied by the quantization matrix to recover the DCT coefficients in the inverse quantization step. Then, an inverse DCT is applied to retrieve the residues, which are added to the prediction made. In this way, the original blocks of the partitioning are retrieved, as explained in Section 4.1.5.

Then, blocks are reassembled to reconstruct the video. Finally, deblocking filters are applied to avoid blocking artifacts on the video.

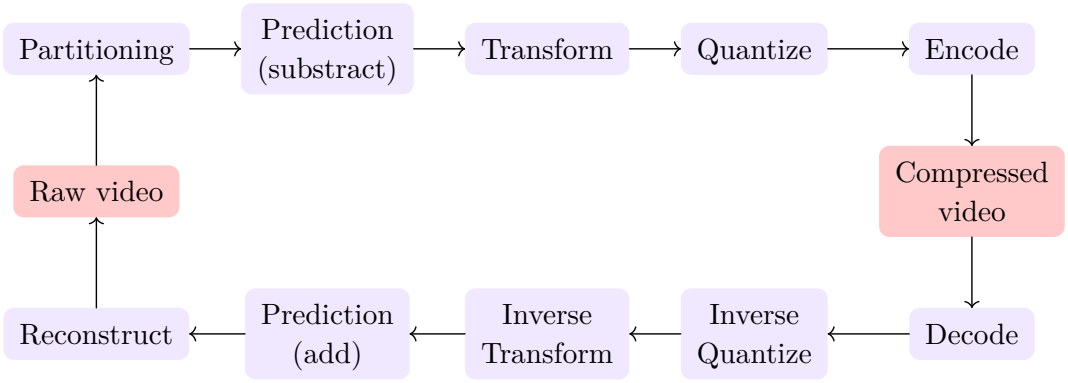


Figure 7: General scheme of video compression/decompression

5 H265

The H265 codec, also known as High Efficiency Video Coding (HEVC), has emerged in 2013 and is still in development. It is based on h264 and improves upon it in terms of compression quality. The H265 codec therefore follows the compression scheme described in Figure 7, with a few modifications that make it more efficient in terms of compression.

The main difference between the H264 and H265 formats is that H264 is designed for single-core processors, while H265 is designed to run in parallel on multi-core processors. The H265 format breaks the image down into tiles that can be processed independently of each other.

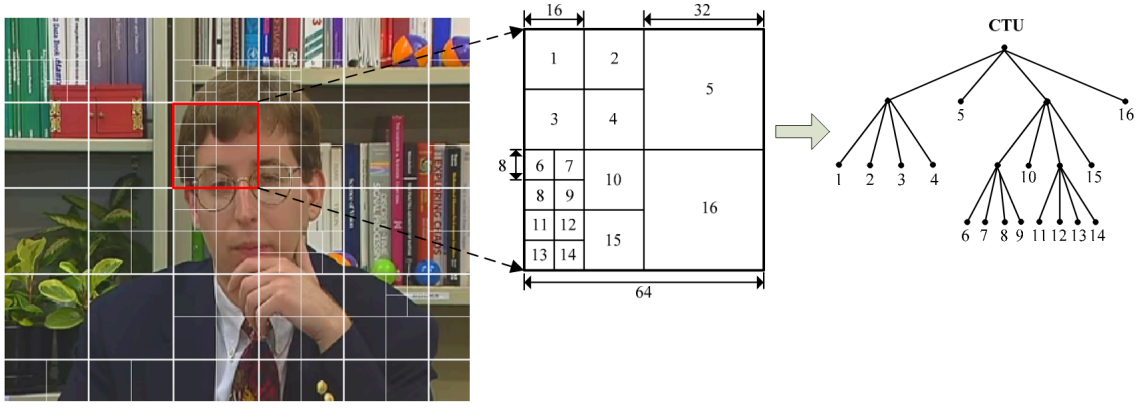
5.1 Compression

The H265 compression follows the same steps than the H264 compression. However, both H265 encoding and decoding is made to work in parallel, allowing less power consumption. But this requires better material, making it not fully compatible with all devices.

5.1.1 Partitioning

This is the main step that makes H265 more efficient than H264. The partitioning structure follows a Quad Tree (QT) structure: instead of working with a decomposition into MacroBlocks, the frame is decomposed in Coding Tree Units (CTU) of size 64×64. This makes compression of flat areas such as blue skies more efficient, because instead of dividing the flat area into 16 blocks of size 16×16, a single block of size 64×64 can be used. Thanks to the larger block size, compression can be performed on higher-quality videos, up to 8K.

Then, each Coding Tree can be divided into 4 sub-coding tree, until a minimum size of 8×8, depending on requirements. The final blocks generated by the Quad Tree structure are called Coding Unit. The Figure 8 show an example of decomposition of a video frame into CTU and CU.



(a) **(b)**

Figure 8: Frame decomposition into CTU **(a)** and quadtree decomposition of a CTU **(b)** [1]

Thanks to Wavefront Parallel Processing (WWP) technology, each Coding Tree Unit can be processed in parallel, reducing computation time on multi-core processors.

5.1.2 Prediction

The prediction is made on Coding Units. During the prediction step, the Prediction Unit (PU) is created. It contains all information necessary to perform the prediction, such as prediction mode used or sub-decomposition of the Coding Unit.

Intra-prediction support handles 35 different modes, compared to 9 for H264, enabling more accurate prediction despite higher computational complexity. This way, the prediction is more accurate, creating a smaller prediction error that can be more compressed.

Inter-frame prediction in H265 is better than in H264 thanks to the flexibility of frame partitioning. In H264, only 16×16 blocks are used for inter-prediction, whereas in H265, block sizes can range from 8×8 to 64×64, allowing for better motion tracking. In addition, H265 can use up to 16 previous frames to compute the motion vector prediction (with always at least an I-frame in the set of frames used). Finally, some novelties are used to increase accuracy of inter-prediction:

- Advanced motion vector prediction. This improves motion vector calculation by creating a list of candidate vectors and choosing the one that gives the best prediction.
- Merge mode. Instead of calculating the motion vector, a block can merge with a neighboring block and inherit its motion information. This is a significant improvement for uniform movements in a video, reducing calculations and the amount of information that needs to be stored.

This is why prediction is more accurate, which results in lower prediction errors that requires less data to be stored. However, prediction requires more computations compared to H264.

5.1.3 Transform

The transformation stage uses the prediction unit to perform a prediction. The difference between the prediction and the original block is then calculated, creating residuals that correspond to the prediction error, as in H264. Similar to H264, this prediction error is then transformed in the DCT domain and stored as a transformation unit (TU). However, compared to H264, H265 uses DCT, but also discrete sinusoidal transform (DST) to perform the decomposition of the coefficients of certain blocks of the luminance channel, which allows for better compression thanks to the structure of these specific blocks.

Finally, quantization is applied as in H264, then H265 uses an improved version of H264′s CABAC entropy coding to store the results.

We can represent the final structure of H265 like in figure blabla.

6 H266

The H266 codec, also known as Versatile Video Coding (VVC) or MPEGi, is the successor to H265. This codec was introduced in 2020 and is still under development. It enables compression of 360-degree and high dynamic range (HDR) videos, supports video compression up to 16K quality, and brings improvements to video compression, making it up to twice as efficient as H265.

6.1 Compression

6.1.1 Partitioning

The main change bring by H266 is its partitioning structure more flexible than the Quad Tree structure of H265: the Multi-Type Tree (MTT) which is a mix-up between Quad Tree, Binary Tree and Ternary Tree. The partition unit is still called a Coding Tree Unit, but can reach a size of 128×128, which is more efficient for high-resolution videos such as 8K or 16K videos.

Each time the frame is divided into a coding sub-tree, the division can be performed as shown on figure Figure 9.

- Binary tree: 1 cut of the area, horizontal or vertical
- Ternary tree: 2 cuts of the area, horizontal or vertical
- Quadratic tree: area divided into 4 quadrants.

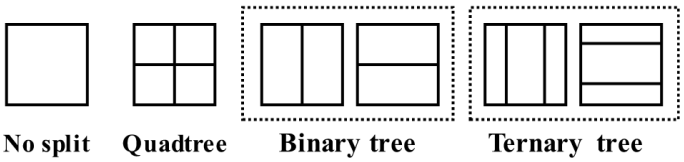


Figure 9: Block decomposition in H266 [2]

The new partitioning system therefore allows for more accurate tracking of object shapes.

6.1.2 Prediction

Instead of having 35 different prediction modes, H266 introduces 67, allowing for greater accuracy during prediction.

In addition, some new tools are introduced for intra-prediction, such as:

- Matrix-based Intra Prediction (MIP). This tool uses a prediction matrix that has been predefined through prior learning.
- Intra Sub-Partition (ISP). The block can be broken down into 2 or 4 sub-blocks, each of which will have its own prediction.
- Cross-Component Linear Model (CCLM). Chrominance prediction is improved by using the existing correlation between chrominance and luminance.

And for inter-prediction:

- Affine Motion Compensation, allowing affine transformation of the block, such as zooming, rotation, etc. This is extremely useful for objects rotating or zooming.

- Decoder-side Motion Vector Refinement (DMVR), a new feature added to the decoder side that allows the decoder to refine the received motion vector. This means that the transmitted motion vector can be of lower quality.
- Bi-directional Optical Flow (BDOF). This type of tool is generally implemented in video editing software such as DaVinci Resolve to avoid jerky slow motion. The principle consists of increasing the prediction based on future and past images. In software, this method is used to add additional images predicted from previous and past images in order to smooth out slow motion.
- Geometric partitioning, which allows the block to be divided into two sub-blocks (for example, by making a diagonal cut) and different movements to be applied to each sub-block. This makes movement tracking more accurate.

6.1.3 Transform

For transformation, since H266 doesn't partition images into blocks, the transformation can also be applied to non-square blocks. The encoder can choose between different transformations, such as Discrete Cosine Transform VIII and Discrete Sinus Transform VII, in order to apply the best transformation for a data compression.

6.1.4 Quantization

Unlike the H265 or H264 codec, the H266 codec uses adaptive quantization, called Dependent Quantization. During the rounding phase of quantization, the coefficient value will be rounded based on the previous coefficient, which will increase the efficiency of entropy coding.

For instance, if we have the sequence $[0.2, 0.3, 0.6]$, normal rounding will return $[0, 0, 1]$. However, dependent quantization will return $[0, 0, 0]$, because for 0.6, it will detect that the previous value was 0, so instead of putting 1, it will put 0, which does not break the existing sequence of 0. The sequence 0 is therefore preserved and increased, which improves the entropic coding of the sequence.

7 AOMedia Video 1 (AV1)

The AOMedia Video 1 (AV1) codec is a royalty-free and open-source codec that has been developed by "Alliance for Open Media", an alliance between Amazon, Google, Netflix, VideoLAN and other actors. The first version was released in 2018 and this codec is still in improvement. This codec was created to success the VP9 Google codec for streaming video on internet and social network and is now used and supported by some range of devices.

The base principle stay the same. However, the method differs from others. First of all, the AV1 remove the video noise, avoiding complex encoding of noise. Then, during decoding, some artificial noise is added to the video to achieve a realist result, allowing better stream speed with approximatively same result than with noise encoding…

7.1 Compression

7.1.1 Partitioning

The frame is decomposed in SuperBlocks (SB). At the begining, the encoder choose between SuperBlocks of size 128×128 or 64×64. Then, each block can be divided in 10 different way, as shown in Figure 10:

- none: the block is not partitioned
- split: partitioning into 4 square sub-blocks
- horizontal: horizontal division of the block
- horizontal 4: the block is divided into 4 equal horizontal strips
- vertical: vertical partitioning of the block
- vertical 4: the block is divided into 4 equal vertical strips
- horizontal A: the block is first divided horizontally, then the upper part is partitioned vertically
- horizontal B: identical to horizontal A, but with the lower part partitioned vertically
- vertical A: the block is first divided vertically, then the left part is partitioned horizontally
- vertical B: identical to vertical A, but with the right sub-block partitioned horizontally

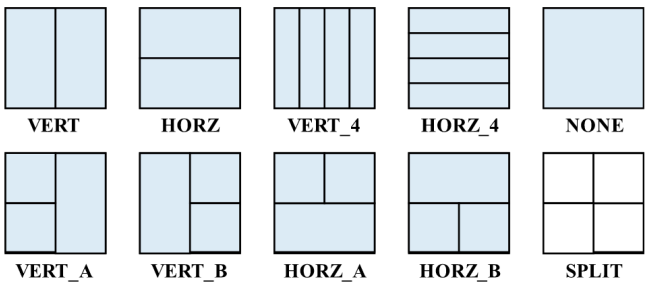


Figure 10: Block decomposition in AV1 [2]

The minimal block size is of 4×4. Thanks to this 10 different way of partitioning the blocks, the AV1 codec is able to effectively detect the shape of complex objects. In addition, thanks to the size of SuperBlocks, high-resolution videos with flat areas such as blue skies are processed efficiently.

Compared to H266 partitioning, the AV1 codec performs less complex partitioning, but thanks to predefined partitioning shapes, partitioning can be performed more efficiently than with H266, and it is easily parallelizable with hardware acceleration already implemented in many devices, such as NVIDIA graphics cards since the 3000 series.

7.1.2 Prediction

The AV1 codec uses up to 56 prediction modes. However, some new tools are used to achieve an accurate prediction, such as for intra-predictions:

- Paeth Predictor: this is an algorithm taken from PNG format that search best prediction according to up, down and diagonal pixels.
- Smooth Predictor: tool optimized for progressive gradients
- Chroma from Luma (CFL): a tool that predicts chrominance from the luminance channel. Since the luminance channel offers better quality, this allows for accurate and high-quality painting of the chrominance channel. This tool is one of the best data savers in AV1.

And for inter-prediction:

- Overlapped Block Motion Compensation (OBMC): this tool consists to take motion vectors of neighboring blocks and mixing them into block's border, allowing smooth block transition.
- Wraped Motion: tool that looks after small affine transformations such as zoom, rotation, etc…
- Compound Prediction: tool that combine multiple reference frame with complex masking. The idea is that certain objects can be better predicted from previous images, for example, and that other objects in the same video can be better predicted from subsequent images. In this way, a mask is created to retain the best prediction for the object and the other object based on the previous and subsequent images.

As with H266, the prediction obtained using the new tools and methods is more accurate, but at the cost of heavier computations.

7.1.3 Transform

Contrary to H265 which is limited to 32×32 transformations, AV1 can achieve transformation on blocks up to 64×64 in size. Since a 2D transformation can be applied first in one dimension and then in the other, AV1 uses a combination of different types of transformations, applied separately to each dimension. The types of transformations used are the following:

- Discrete Cosine Transform (DCT): it is the classical transformation also used in H264, H265 and H266.
- Asymmetric Discrete Sine Transform (ADST): this transformation is particularly efficient for directional gradients.
- FlipADST: it's only the ADST applied in inverse order (right to left or down to up). So with ADST and FlipADST, all directional gradients are well managed.
- Identity (IDTX): no transformation. This is very usefull for brutal transitions like black text on white, avoiding some artifacts introduced by a transformation such as blurring edge or something else.

Thanks to this 16 combinations of transformations, information is better preserved in quantization step.

7.1.4 Quantization

The quantization used in AV1 is a granular and dynamic quantization based on segmentation and ΔQ.

In fact, AV1 quantization segments frames into a segment map based on the visual interest of each segment. Then, a ΔQ is added to each segment depending on the visual interest of the given segment in order to make the final quantization more or less accurate. In this way, granular quantification will perform a perceptual quantization that will preserve the most important video's information for human visual system. For instance, a uniform blue sky will have stronger quantization than the actor's face thereby assigining fewer bits to the low-detail area than to the high-detail area.

7.1.5 Entropy encoding

Entropy encoding uses Asymmetric Numeral Systems (ANS) to perform encoding, which is as efficient as the CABAC encoding used in other codecs. However, this method is faster than the classic CABAC method thanks to easier parallelization on modern architectures and direct encoding of symbols instead of converting them to bits before encoding.

8 Forensic

Now that we know how video compression works for different codecs, we will examine the artifacts generated by these different codecs to give an idea of how digital forensics can exploit compressed video to recover its history.

When a video is modified, it must be decoded, modified and finaly reencoded as shown on Figure 11.

Figure 11: Modification of a video

Many digital investigation methods are well known for the H264 and H265 formats. But as the H266 and AV1 formats are still emerging, it's more complex to properly analyze videos encoded with these codecs, especially since they involve complex video compression methods. However, some studies have already been conducted on these codecs such as the detection of double compression for H266 codec [3].

Even if certain camouflage techniques are used to hide some video modifications [4], digital forensics experts remain up to date thanks to the efforts of the deep learning network used for video forensics analysis [5], [6]

8.1 Partitioning

The partitioning of the frames into both MacroBlock, SuperBlocks and Coding Tree Units produces some artifacts.

8.1.1 H264

In the H264 format, macroblock partitioning introduces a rigid grid that can be exploited to detect, for instance, double compression with video cropping or double compression with an object added to the video.

This causes artifacts from the old grid due to incorrect alignment of the previous grid with the grid created by recompression, such as discontinuities introduced by DCT at the old blocks border. Furthermore, if recompression is performed with another codec, the artifacts generated by the H264 grid will still appear, making it possible to detect transcoding [1].

8.1.2 H265

The H265 Coding Tree structure is more complex than the H264 fixed grid, but still have some artifacts. In fact, different encoders, such as NVENC, x265, or those used by software, do not all partition frame in the same way, which sometimes makes it possible to detect which encoder or software was used. It is also possible to detect certain deepfake videos, as the generated videos can introduce noise into a flat area, making the partitioning of images nonsensical (for example, the sky broken down into 4 x 4 CU).

8.1.3 H266

The H266 coding tree introduces unique artifacts due to its complex and non-square decomposition. In fact, if the video appears to come from a given camera but the MTT partitioning is perfectly executed, this means that the partitioning was not performed by the given camera, as this requires a lot of computation which cannot be performed by the camera.

8.1.4 AV1

Since the AV1 format removes noise before encoding, all blocks are very clean, which is suspicious. In addition, each platform such as YouTube, Netflix, etc. uses a different partitioning complexity, favoring speed (in the case of YouTube) or quality. In this way, a study of the complexity of partitioning can provide information about the platform from which the video originates. Furthermore, the specific shapes introduced by partitioning may indicate that it was not performed with H264 or H265, which only work with square blocks.

8.2 Prediction

This is the stage of video compression that produces the most interesting artifacts and provides the most useful information for digital forensics.

For instance, it is possible to detect if a video is generated by AI or is real by a study of motion vectors computed during inter-prediction step: indeed, the generated video will present certain inconsistencies in its motion vectors compared to a real video that can be detected [7]. In this way, a generated video that appears increasingly realistic can be detected as a generated video rather than a real video, which can be very useful if the generated video involves sensitive or political information or other matters.

Furthermore, due to the limited resources and computing power of certain devices such as cameras, they do not support all prediction modes for complex encoders that contain several prediction modes such as H265, H266, or AV1. These characteristics can be used to identify the source of a given video, although hackers can simulate the source by forcing the use of certain prediction modes only.

Finally, the main feature used to perform digital forensic analysis on a video is the Group Of Pictures structure created by inter-prediction, as shown in Figure 6.

In general, during video acquisition, the video is encoded only in I-frames and P-frames, since B-frames depend on subsequent frames that have not yet been acquired and the computation of B-frames is more complex, drastically reducing the battery performance. Recompressing this kind of video will result in the conversion of different frames into another type of frame, as shown by Figure 12. These conversions leave some artifacts that can be detected and analysed [8].

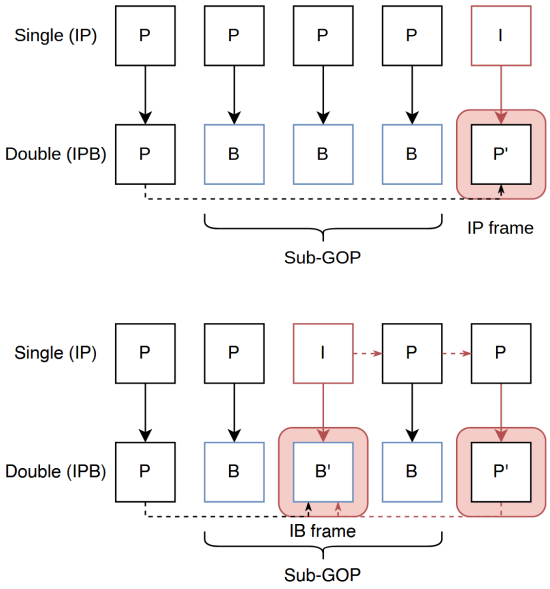


Figure 12: Changing frame type during recompression

In addition, when some changes are made to the video, such as inserting, deleting, or duplicating frames, the video's GOP is modified as shown on Figure 13.

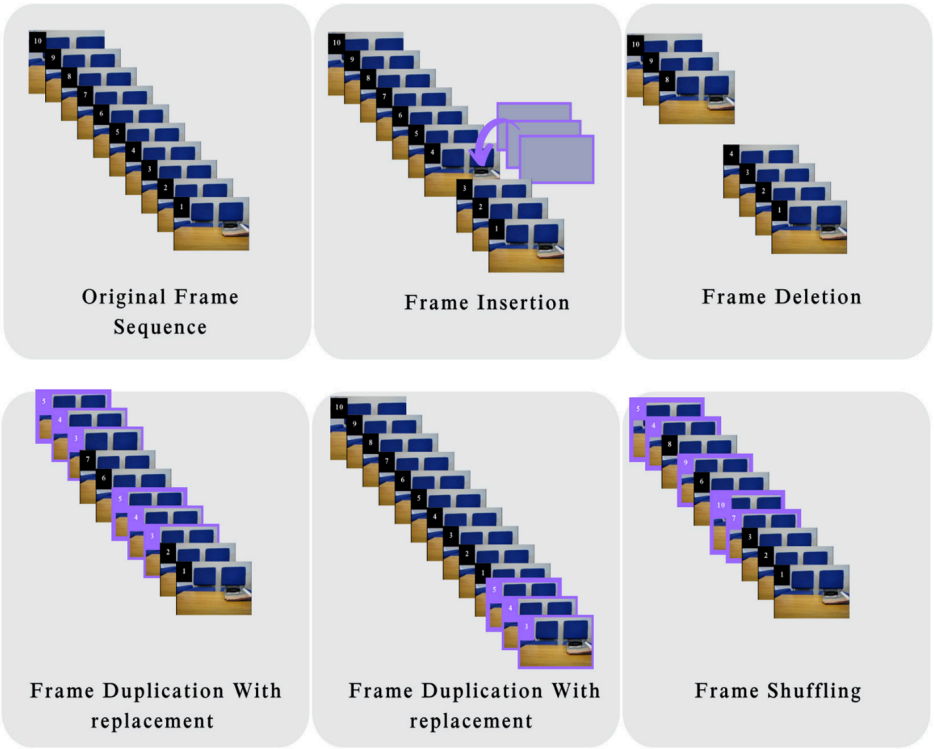


Figure 13: Video modification [9]

These types of video manipulations are common, for example to remove a person from a surveillance camera. That's why it is important to detect this kind of manipulation. When this type of modification is made to a video, the continuity of the GOP structure is generally broken, which can be detected by the Earth's Mover distance [9]. This can also be detected

in the event of recompression, as there will be a conversion of the frame type similar to that shown in the Figure 13.

However, it is becoming increasingly difficult to detect video modifications if they are made using camouflage techniques [4].

8.3 Quantization and Entropy encoding

The quantization step produce some artifacts such as blocking, ringing or flickering artifacts [10].

- Blocking artifacts: These artifacts are caused by DCT transformation and block quantization, which create discontinuities at the block boundaries.
- Ringing artifacts: The quantization create some distortions around sharp borders by removing high frequencies.
- Flicker: this is a temporal artifact that manifests itself as rapid discontinuities in luminance, particularly in flat, stationary areas. This phenomenon is due to a difference in the loss of DCT coefficients caused by quantization rounding in consecutive images, but also to a temporal discontinuity between I-frames and the preceding P and B-frames [11]. In fact, I-frames are encoded without information about the other frames, as explained in Section 4.1.4.2. I-frames are therefore subject to quantization artifacts and rounding errors. Furthermore, if we consider a device such as a camera, B-frames are not created as explained above due to the computation time and live encoding of the acquired video. The video is therefore composed solely of I-frames and P-frames. And since P-frames are based on the previous I-frame, this means that when a new I-frame appears, there is a discontinuity between the last P-frame and the new I-frame due to quantization artifacts and rounding errors, which are not related.

However, these artifacts are becoming less and less common thanks to improved methods. For example, in AV1, if there is a sharp edge, no quantization is performed in order to avoid resonance artifacts. In addition, the use of B-frames reduces flickering by taking the previous and following frames, allowing for a smoother transition between the P-frame and the following I-frame. But they are not completely eliminated, and the use of machine learning can help detect them.

In the binary stream created by encoding compressed video, certain metadata can be exploited to detect the software used for editing or the type of device used to capture the video, as well as other important information [12]. In addition, information such as motion vectors is encoded in the bitstream. This makes analyzing inter-frame modification or motion vectors more efficient and less memory-intensive, while avoiding video decoding [13].

9 Conclusion

9.0.1 Note on the use of AI

- Deepl for English correction
- ChatGPT and Gemini for understanding some concepts and summarizing some articles
- Perplexity and SciSpace for document research

Bibliography

[1] Z. Zhang, C. Liu, Z. Li, L. Yu, and H. Yan, "Detection of Transcoding from H.264/AVC to HEVC Based on CU and PU Partition Types," *Symmetry*, vol. 11, no. 11, p. 1343, Nov. 2019, doi: 10.3390/sym11111343.

[2] "(PDF) AV1 and VVC Video Codecs: Overview on Complexity Reduction and Hardware Design," *ResearchGate*, doi: 10.1109/OJCAS.2021.3107254.

[3] Q. Xu, D. Xu, H. Wang, Z. Mi, Z. Wang, and H. Yan, "Detecting Double H.266/VVC Compression with the Same Coding Parameters," *Neurocomputing*, vol. 514, pp. 231–244, Dec. 2022, doi: 10.1016/j.neucom.2022.09.153.

[4] P.-C. Su, P.-L. Suei, M.-K. Chang, and J. Lain, "Forensic and Anti-Forensic Techniques for Video Shot Editing in H.264/AVC," *J. Vis. Comun. Image Represent.*, vol. 29, no. C, pp. 103–113, May 2015, doi: 10.1016/j.jvcir.2015.02.006.

[5] V. T. R., A. Ajina, H. Dongare, S. Joshi, G. L. S. Varma, and R. D. Rathod, "Video Forensic Analysis Using Deep Learning Models," in *2025 International Conference on Emerging Technologies in Computing and Communication (ETCC)*, June 2025, pp. 1–6. doi: 10.1109/ETCC65847.2025.11108521.

[6] Q. Bao, Y. Wang, H. Hua, K. Dong, and F. Lee, "An Anti-Forensics Video Forgery Detection Method Based on Noise Transfer Matrix Analysis," *Sensors*, vol. 24, no. 16, p. 5341, Jan. 2024, doi: 10.3390/s24165341.

[7] P. Grönquist, Y. Ren, Q. He, A. Verardo, and S. Süsstrunk, "Efficient Temporally-Aware DeepFake Detection Using H.264 Motion Vectors," no. arXiv:2311.10788. arXiv, Feb. 2024. doi: 10.48550/arXiv.2311.10788.

[8] Y. Furushita, D. Baracchi, M. Fontani, D. Shullani, and A. Piva, "Detection of Double Compression in HEVC Videos Containing B-Frames," *Journal of Imaging*, vol. 11, no. 7, p. 211, July 2025, doi: 10.3390/jimaging11070211.

[9] M. M. Ali, N. I. Ghali, H. M. Hamza, K. M. Hosny, E. Vrochidou, and G. A. Papakostas, "Interframe Forgery Video Detection: Datasets, Methods, Challenges, and Search Directions," *Electronics*, vol. 14, no. 13, p. 2680, Jan. 2025, doi: 10.3390/electronics14132680.

[10] C. R. Anil and D. B. R. Krishna, "H.264 VIDEO CODING ARTIFACTS- MEASUREMENT AND REDUCTION OF FLICKERING."

[11] Y. Kuszpet, D. Kletsel, Y. Moshe, and A. Levy, "POST-PROCESSING FOR FLICKER REDUCTION IN H.264/AVC."

[12] Z. Xiang, J. Horváth, S. Baireddy, P. Bestagini, S. Tubaro, and E. J. Delp, "Forensic Analysis of Video Files Using Metadata," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, June 2021, pp. 1042–1051. doi: 10.1109/CVPRW53098.2021.00115.

[13] H. Jean, E. Giguet, and C. Charrier, "Video Forgery Detection by Bitstream Analysis," in *CEUR Workshop Proceedings of Colour and Visual Computing Symposium 2022 (CVCS 2022)*, A. S. Sole and D. Guarnera, Eds., Gjøvik, Norway, Sept. 2022.